

MOLLY MARKET

SYSTÈME DE GESTION INTÉGRÉ & POINT DE VENTE SUPERMARCHÉ

DOSSIER TECHNIQUE D'ARCHITECTURE & GUIDE DE PRÉSENTATION ORALE (SOUTENANCE)

PÉRIMÈTRE & CONTENU DU DOCUMENT :

- Architecture 3-Tiers : React 19 (SPA) + Express (Node.js) + PostgreSQL 16+
- Moteur Métier 100% Procédural : Procédures PL/pgSQL, Triggers & Vues
- Fonctionnalités Supermarché : Mode Offline-First, ESC/POS 80mm & Multi-Caisses
- Sécurité & Fiabilité : Authentification JWT, RBAC, Index B-Tree & Sauvegardes 7j
- Scénarios Démonstration Pas-à-Pas & Fiches Questions/Réponses pour le Jury
- Monnaie & Comptabilité : Francs CFA (FCFA) • Conformité OHADA / Côte d'Ivoire

Supermarché Molly Market Abidjan • République de Côte d'Ivoire

Promotion Informatique & Systèmes d'Information • Année 2026

Document confidentiel d'ingénierie et de soutenance

1. PRÉSENTATION DU SYSTÈME ET STACK TECHNIQUE

Molly Market est une solution de gestion de supermarché de grade entreprise conçue selon une architecture 3-Tiers découplée et orientée procédures. La règle d'or du système est le zéro calcul arbitraire côté frontend : l'ensemble des calculs de stocks, TVA, totaux de caisse, et billettages sont scellés au cœur de PostgreSQL.

Composant / Couche	Technologies Utilisées	Rôle & Bénéfice Opérationnel
Couche Présentation (Frontend)	React 19, TypeScript 5.8, Vite 6.2, Tailwind CSS, Zustand, Lucide React	Interface caisse POS réactive, formulaires de gestion, visualisations KPI et exports.
Mode Hors-Ligne (Offline-First)	IndexedDB API (mollymarket_pos_offline), Hook useOnlineStatus	Continuité des encaissements en cas de coupure réseau + synchronisation automatique.
Impression Caisse Thermique	ESC/POS Protocol (80mm / 58mm), WebSerial / WebUSB API	Impression instantanée sur rouleau thermique sans dialogue navigateur avec coupe papier.
Couche Service (Backend API)	Node.js 22 LTS, Express.js 4.21, pg.Pool (node-postgres)	Passerelle HTTP stateless légère, requêtes paramétrées (\$1, \$2) et sécurité.
Sécurité & Contrôle d'Accès	JSON Web Tokens (JWT), Middleware RBAC (Directeur, Vendeur, etc.)	Chiffrement des sessions, signature d'accès et protection des routes d'administration.
Couche Données & Métier	PostgreSQL 16+, PL/pgSQL, Triggers, Vues Métier, Index B-Tree	Exécution transactionnelle ACID, audit trail automatique et procédures d'encaissement.
Sauvegardes & Résilience	Script pg_dump / backup.js avec rétention tournante 7 jours	Sauvegardes SQL horodatées automatiques avec suppression cyclique des anciens dumps.

2. FLUX TRANSACTIONNEL & GESTION HORS-LIGNE (OFFLINE-FIRST)

1. Scan & Panier	Le caissier scanne les articles (code-barres). Calcul visuel TTC/HT instantané côté client.
2. Validation En Ligne	Appel POST /api/ventes avec Bearer JWT -> Exécution CALL effectuer_vente(...) -> Décrément automatique du stock par trigger -> Réponse immédiate avec ticket officiel.
3. Coupure Réseau (Offline)	Si le serveur est injoignable, la vente est stockée dans IndexedDB (pending_sales). Un ticket provisoire TK-OFF-XXXXXX est émis sans bloquer le passage des clients.
4. Synchronisation Auto	Dès reconnexion, le hook useOnlineStatus pousse les ventes en attente vers PostgreSQL et affiche un toast de confirmation.

1. STRUCTURE DES TABLES POSTGRESQL & INTÉGRITÉ

Nom de la Table	Colonnes Principales	Rôle Métier & Relations
utilisateurs	id, matricule, nom, prenom, email, mot_de_passe, role, telephone, actif	Gestion des comptes et des rôles (Directeur, Administrateur, Vendeur, Magasinier).
caisses	id, code, nom, emplacement, statut, derniere_activite	Terminaux physiques de vente pour gestion multi-caisses simultanée (Centrale, Express, Étage).
categories	id, nom, description, icone, couleur, actif	Rayons du supermarché (Alimentation, Boissons, Entretien, Vêtements, Cosmétiques...).
produits	id, code_barre, nom, categorie_id, prix_vente, prix_achat, stock_actuel, seuil_alerte	Catalogue articles avec codes-barres EAN-13, prix en FCFA et suivi de stock en temps réel.
ventes	id, numero_ticket, client_id, vendeur_id, caisse_id, date_vente, montant_total, statut	En-tête des tickets de caisse avec référence unique TK-YYYYMMDD-XXX et caisse d'émission.
lignes_vente	id, vente_id, produit_id, quantite, prix_unitaire, montant_total	Détail des articles vendus. L'insertion déclenche le trigger de décrémentation de stock.
paiements	id, reference_paiement, vente_id, montant, mode_paiement, date_paiement, statut	Encaissements ventilés par mode : especes, wave, orange_money, mtn_money, carte, cheque.
points_caisse	id, numero_session, caisse_id, date_journee, heure_ouverture, statut, fond_initial	Sessions journalières de caisse (statuts : ouverte, soumise_directeur, validee_directeur).
billetage_point_caisse	id, point_caisse_id, mode_paiement, montant_theorique, montant_compte, ecart	Réconciliation et constat d'écart par mode de règlement (Espèces vs Mobile Money).
mouvements_stock	id, produit_id, type_mouvement, quantite, stock_avant, stock_apres, motif	Traçabilité immuable (Audit Trail) de toutes les entrées/sorties de stock.
fournisseurs / achats	id, code_fournisseur, nom_entreprise, telephone, adresse / lignes_achat	Circuit d'approvisionnement : commande magasinier -> réception stock -> paiement directeur.

2. PLAN D'INDEXATION B-TREE (PERFORMANCES < 10MS)

idx_ventes_date & idx_ventes_caisse	ventes(date_vente), ventes(caisse_id)	Accélération des calculs de CA journalier et filtrage des sessions de caisse.
idx_lignes_vente_vente / produit	lignes_vente(vente_id, produit_id)	Jointure instantanée des lignes d'articles lors de l'édition des tickets.
idx_paiements_mode & date	paiements(mode_paiement, date_paiement)	Ventilation instantanée des encaissements Mobile Money (Wave, OM) et Espèces.
idx_mouvements_produit_date	mouvements_stock(produit_id, date_mouvement)	Reconstitution rapide de l'historique d'un article lors des audits de stock.

1. PROCÉDURES STOCKÉES & LOGIQUE MÉTIER TRANSACTIONNELLE

Nom Procédure	Signature PL/pgSQL	Logique Exécutée & Règles de Gestion
effectuer_vente(...)	CALL effectuer_vente(client_id, p_lignes, vendeur_id, mode_paiement, p_vente_id, p_numero_ticket, p_montant_total)	1. Génère un numéro de ticket TK-YYYYMMDD-XXX. 2. Insère dans `ventes`. 3. Boucle sur le JSON des articles et insère dans `lignes_vente`. 4. Le trigger `trg_decrementer_stock_vente` décrémente automatiquement le stock et vérifie la disponibilité. 5. Insère le paiement dans `paiements` avec référence PAY-YYYYMMDD-XXX.
ajuster_stock(...)	CALL ajuster_stock(produit_id, nouvelle_quantite, motif, utilisateur_id)	Rectifie le stock physique d'un article après inventaire et crée une ligne dans `mouvements_stock` avec le motif.
reception_stock(...) & payer_facture(...)	CALL reception_stock(achat_id, user_id) / CALL payer_facture_fournisseur(achat_id, user_id)	Incrémente les stocks reçus du fournisseur et valide la facture pour décaissement par la direction.

2. TRIGGERS AUTOMATIQUES ET VUES MÉTIER

Nom Déclencheur / Vue	Type d'Événement / Cible	Règle de Gestion Assurée
trg_decrementer_stock_vente	AFTER INSERT ON lignes_vente	Vérifie si stock_actuel >= quantite. Décrémente le stock et génère un mouvement de type 'vente'. En cas de stock insuffisant, lève une exception SQL qui annule toute la vente.
trg_audit_suppression	BEFORE DELETE ON produits	Interdit la suppression physique des produits ayant des ventes rattachées et journalise l'action dans `journal_suppressions`.
vue_stock / vue_commandes	Vues relationnelles jointes	Vue_stock expose le stock actuel et le statut d'alerte. Vue_commandes joint les clients, vendeurs et totaux.
vue_statistiques / top_produits	Vues décisionnelles	Agrège le CA journalier, mensuel, la répartition par rayon et le palmarès des articles les plus vendus.

SCÉNARIOS DE DÉMONSTRATION À EFFECTUER LORS DE LA PRÉSENTATION

Scénario Métier	Actions Pas-à-Pas à Réaliser Devant le Jury
Scénario 1 : Authentification & Multi-Caisses	1. Se connecter avec le compte Caissier (Noam Koffi, role: Vendeur). 2. Sélectionner le terminal physique dans le POS (ex: Caisse Principale N°1). 3. Constater l'indicateur vert Ø=ßà "Connecté au serveur" et la présence d requêtes.
Scénario 2 : Encaissement & Décrémentation Stock	1. Scanner 2 articles (ex: Lait Bonnet Rouge + Riz Dinor) via le scanner ou les boutons de bip rapide. 2. Sélectionner le mode de règlement Espèces ou Wave Mobile Money. 3. Valider l'encaissement : constater le déclenchement de la procédure SQL `effectuer_vente`, la décrémentation immédiate du stock dans `produits` et l'apparition du ticket. 4. Cliquer sur "Impression Thermique 80mm (ESC/POS)" pour montrer le format de ticket de caisse direct.
Scénario 3 : Simulation Hors-Ligne (Offline-First)	1. Couper momentanément le serveur backend dans le terminal. 2. Constater le basculement instantané du badge en Ø=ßà "Mode Hors-Lign 3. Effectuer une vente : elle est enregistrée dans IndexedDB et émet un ticket local TK-OFF-XXXXXX. 4. Relancer le serveur backend : le hook détecte la reconnexion et synchronise automatiquement la vente.
Scénario 4 : Clôture de Caisse & Billetage	1. Aller sur l'onglet "Point de Caisse" (journée en cours). 2. Constater les montants théoriques calculés par PostgreSQL pour chaque moyen (Espèces, Wave, OM). 3. Saisir le comptage physique dans le tiroir-caisse -> Constater l'écart 4. Soumettre la caisse au Directeur : la session est verrouillée jusqu'au lendemain. 5. Se connecter en Directeur (Eden Touré) pour valider officiellement la session et exporter le PV en PDF.
Scénario 5 : Sauvegarde de Base de Données	1. Exécuter le script `node server/scripts/backup.js`. 2. Montrer la création du dump horodaté dans `server/backups/` et la règle de rétention 7 jours.

QUESTIONS CLASSIQUES DU JURY ET ARGUMENTAIRE TECHNIQUE

Question Prévisible du Jury	Réponse Technique Recommandée
Pourquoi avoir choisi PostgreSQL avec des procédures stockées plutôt que de tout calculer dans Node.js ?	Pour garantir l'intégrité absolue des données et le principe ACID. Si la logique était dans Node.js, une coupure de courant ou un crash applicatif au milieu d'une vente pourrait enregistrer le paiement sans décrémenter le stock. Avec PostgreSQL et les procédures stockées transactionnelles (BEGIN...COMMIT), toute la vente et ses impacts de stocks sont atomiques : tout réussit ou tout est annulé.
Comment le système gère-t-il la concurrence si 3 caisses vendent le dernier article en même temps ?	PostgreSQL applique un verrouillage de ligne (Row-Level Locking). Le trigger <code>`trg_decrementer_stock_vente`</code> vérifie la condition <code>`stock_actuel >= quantite`</code> . La première caisse valide la décrémentation, et les deux autres déclenchent une exception SQL "Stock insuffisant", empêchant tout sur-stockage négatif.
Comment fonctionne le mode hors-ligne sans connexion internet ?	Le frontend exploite l'API IndexedDB standard du navigateur (base locale <code>`mollymarket_pos_offline`</code>). Lors d'une coupure réseau, la transaction est stockée localement dans la file d'attente <code>`pending_sales`</code> . Dès le rétablissement de la connexion, le hook <code>`useOnlineStatus`</code> dépile la file et transmet les ventes au backend de manière transparente.
Quelle est la stratégie de sécurité mise en place sur les API ?	1. Authentification par jeton JWT signé (expiration 12h). 2. Contrôle d'accès RBAC (Role-Based Access Control) côté backend : un caissier ne peut pas accéder aux routes de sauvegarde ou de suppression. 3. Requêtes SQL 100% paramétrées éliminant tout risque d'injection SQL.
Comment sont gérés les écarts de caisse en fin de journée ?	Le système calcule la vérité théorique pour chaque mode de règlement (Espèces + fond de caisse initial, Wave, Orange Money). Le caissier saisit son comptage réel. La différence génère l'écart exact. La session est verrouillée et transmise au Directeur Eden qui est la seule autorité habilitée à valider ou rejeter le point de caisse.